

Piotr's TechBlog

Java, Spring, Kotlin, microservices, Kubernetes, containers

Getting Started with Spring AI Function Calling

AI, Spring Boot

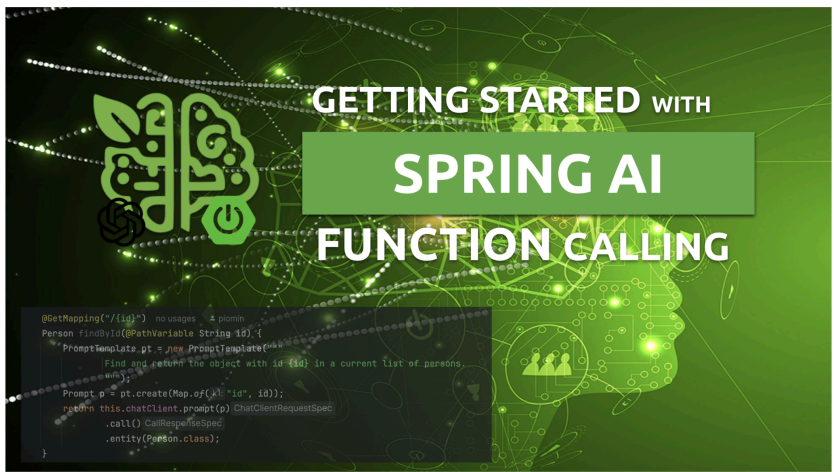
Getting Started with Spring AI Function Calling

By [piotr.minkowski](#) • January 30, 2025 •  6

Search

Follow Blog via Email

Enter your email address to follow this
blog and receive notifications of new



This article will show you how to use Spring AI support for Java function calling with the OpenAI chat model. The Spring AI function calling feature lets us connect the LLM capabilities with external APIs or systems. OpenAI’s models are trained to know when to call a function. We will work on implementing a Java function that takes the call arguments from the AI model and sends the result back. Our main goal is to connect to the third-party APIs to provide these results. Then the AI model uses the provided results to complete the conversation.

This article is the second part of a series describing some of the AI project’s most notable features. Before reading on, I recommend checking out my introduction to Spring AI, which is available [here](#). The first part describes such features as prompts, structured output, chat memory, and built-in advisors. Additionally, it demonstrates the capability to switch between the most popular AI chat model API providers.

Source Code

If you would like to try it by yourself, you may always take a look at my source code. To do that, you must clone my sample GitHub [repository](#). Then you should only follow my instructions.

Problem

Whenever I create a new article or example related to AI, I like to define the problem I’m trying to solve. The problem we will solve in this exercise is visible in the following prompt template. I’m asking the AI model about the value of my stock wallet. However, the model doesn’t know how many shares I have, and can’t get the latest stock

posts by email.

Enter your email address

Follow

Categories

- AI (18)
- Cloud (42)
- Containers (67)
- Continuous Integration (37)
- Data Grids (7)
- Kotlin (9)
- Kubernetes (115)
- Message Brokers (27)
- Micronaut (12)
- Microservices (64)
- Other (28)
- Performance (28)
- Quarkus (18)
- Security (26)
- Spring Boot (113)

prices. Since the OpenAI model is trained on a static dataset it does not have direct access to the online services or APIs.

```
PromptTemplate pt = new PromptTemplate("""
What's the current value in dollars of my wallet based on the latest stock daily prices ?
Return values per each company and a total value as sum of values per each company in JSON format.
""");
```

So, in this case, we should provide private data with our wallet structure and “connect” our model with a public API that returns live stock market data. Let’s see how we tackle this challenge with Spring AI function calling.

Create Spring Functions

WalletService Supplier

We will begin with a source code. Then we will visualize the whole process on the diagram. Spring AI supports different ways of registering a function to call. You can read more about it in the Spring AI docs [here](#). We will choose the way based on plain Java functions defined as beans in the Spring application context. This approach allows us to use interfaces from the `java.util.function` package such as `Function`, `Supplier`, or `Consumer`. Our first function takes no input, so it implements the Supplier interface. It just returns a list of shares that we currently have. It obtains such information from the database through the Spring Data `WalletRepository` bean.

```
public class WalletService implements Supplier<Wallet>

    private WalletRepository walletRepository;

    public WalletService(WalletRepository walletRepos
        this.walletRepository = walletRepository;
    }

    @Override
    public WalletResponse get() {
        return new WalletResponse((List<Share>) walle
    }
}
```

Information about the number of owned shares is stored in the `share` table. Each row contains a company name and the quantity of that

○ Spring Cloud (51)

○ Uncategorized (1)

Tags

Apache Kafka

argocd

Consul

Continuous Delivery

Continuous Integration

Docker

Eureka

gitops

Helm

istio

Java

Jenkins

Jib

JPA

kafka

kind

Knative

Kotlin

Kubernetes

Maven

Micronaut

microservices

Minikube

OAuth2

open-ai

openshift

Performance

postgresql

Prometheus

quarkus

Reactor

REST

security

Skaffold

Spring

spring-ai

Spring Boot

Spring Boot Actuator

Spring Cloud

Spring Cloud Gateway

Spring Cloud Kubernetes

company shares.

```
@Entity
public class Share {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String company;
    private int quantity;

    // ... GETTERS/SETTERS
}
```

Spring Data

testcontainers

testing

Vault

The Spring Boot application launches an embedded, in-memory database and inserts test data into the stock table. Our wallet contains the most popular companies on the U.S. stock market, including Amazon, Meta, and Microsoft.

```
insert into share(id, company, quantity) values (1, '
insert into share(id, company, quantity) values (2, '
insert into share(id, company, quantity) values (3, '
insert into share(id, company, quantity) values (4, '
insert into share(id, company, quantity) values (5, '
```

StockService Function

Our second function takes an input argument and returns an output. Therefore, it implements the **Function** interface. It must interact with live stock market API to get the current price of a given company share. We use the api.twelvedata.com service to access stock exchange quotes. The function returns a current price wrapped by the **StockResponse** object.

```
public class StockService implements Function<StockRe

    private static final Logger LOG = LoggerFactory.g

    @Autowired
    RestTemplate restTemplate;
    @Value("${STOCK_API_KEY}")
    String apiKey;

    @Override
```

```

    public StockResponse apply(StockRequest stockRequest) {
        StockData data = restTemplate.getForObject("http://localhost:8080/stock",
            StockData.class,
            stockRequest.company(),
            apiKey);
        DailyStockData latestData = data.getValues().get(0);
        LOG.info("Get stock prices: {} → {}", stockRequest.company(), latestData);
        return new StockResponse(Float.parseFloat(latestData.getPrice()));
    }
}

```

Here are the Java records for request and response objects.

```

public record StockRequest(String company) { }

public record StockResponse(Float price) { }

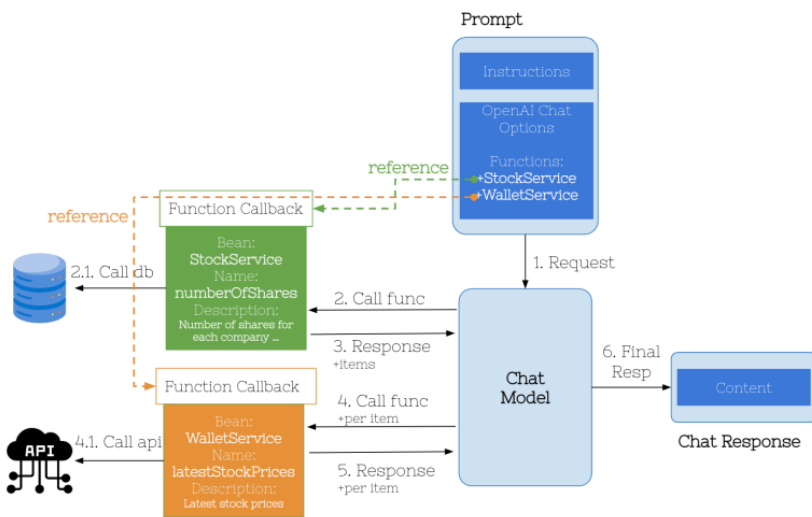
```

To summarize, the first function accesses the database to get owned shares quantity, while the second function communicates public API to get the current price of a company share.

Spring AI Function Calling Flow

Architecture

Here's the diagram that visualizes the flow of our application. The Spring AI **Prompt** object must contain references to our function beans. This allows the OpenAI model to recognize when a function should be called. However, the model does not call the function directly but only generates JSON used to call the function on the application side. Each function must provide a name, description, and signature (as JSON schema) to let the model know what arguments it expects. We have two functions. The **StockService** function returns a list of owned company shares, while the second function takes a single company name as the argument. This is where the magic happens. The chat model should call the **StockService** function for each object in the list returned by the **WalletService** function. The final response combines results received from both our functions.



Implementation

To implement the flow visualized above we must register our functions as Spring beans. The method name determines the name of the bean in the Spring context. Each bean declaration should also contain a description, which helps the model to understand when to call the function. The `WalletResponse` function is registered under the `numberOfShares` name, while the `StockService` function under the `latestStockPrices` name. The `WalletService` doesn't take any input arguments, but injects the `WalletRepository` bean to interact with the database.

```
@Bean
@Description("Number of shares for each company in my
public Supplier<WalletResponse> numberOfShares(Wallet
return new WalletService(walletRepository);
}
```

```
@Bean
@Description("Latest stock prices")
public Function<StockRequest, StockResponse> latestSt
return new StockService();
}
```

Finally, let's take a look at the REST controller implementation. It exposes the `GET /wallet` endpoint that communicates with the OpenAI chat model. When creating a prompt we should register both our functions using the `OpenAiChatOptions` class and its `function` method. The reference contains only the function `@Bean` name.

```

@RestController
@RequestMapping("/wallet")
public class WalletController {

    private final ChatClient chatClient;

    public WalletController(ChatClient.Builder chatCl
        this.chatClient = chatClientBuilder
            .defaultAdvisors(new SimpleLoggerAdvi
            .build();
    }

    @GetMapping
    String calculateWalletValue() {
        PromptTemplate pt = new PromptTemplate("""
        What's the current value in dollars of my wal
        """);

        return this.chatClient.prompt(pt.create(
            OpenAiChatOptions.builder()
                .function("numberOfShares")
                .function("latestStockPrices"
                .build()))
            .call()
            .content();
    }
}

```



Run and Test the Spring AI Application

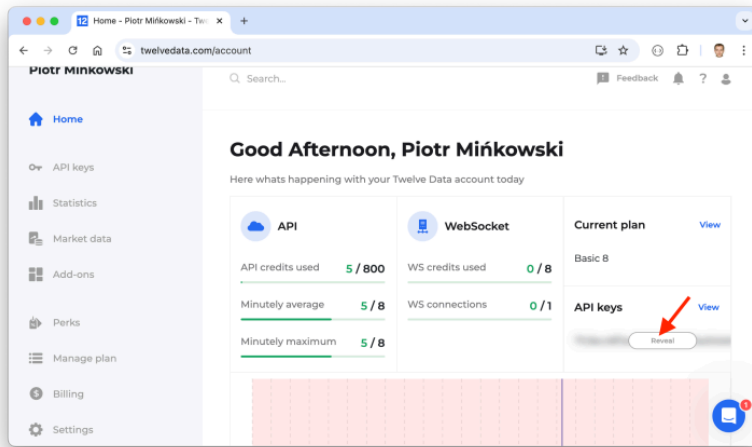
Before running the app, we must export OpenAI and Twelvedata API keys as environment variables.

```

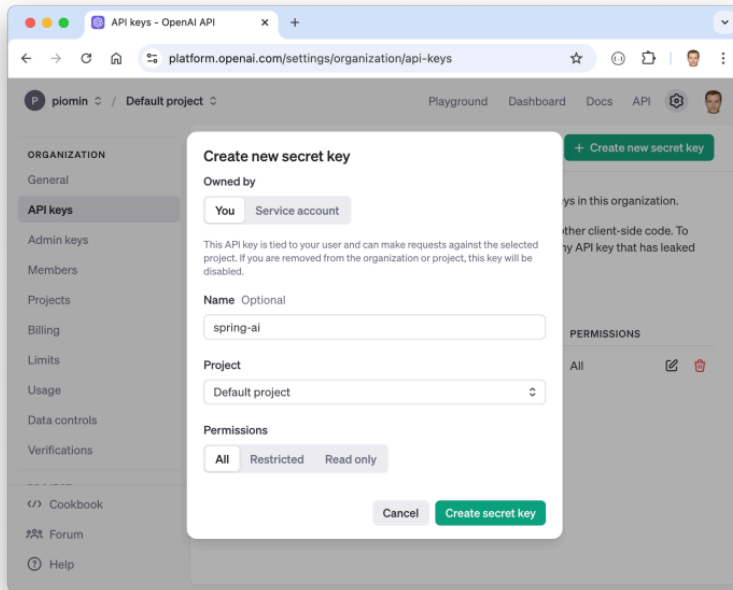
export STOCK_API_KEY=<YOUR_TWELVEDATA_API_KEY>
export OPEN_AI_TOKEN=<YOUR_OPENAI_TOKEN>

```

We must create an account on the **Twelvedata** platform to obtain its API key. The Twelvedata platform provides API to get the latest stock prices.



Of course, we must have an API key on the OpenAI platform. Once you create an account there you should go to that [page](#). Then choose the name for your token and copy it after creation.



Then, we run our Spring AI app using the following Maven command:

```
mvn spring-boot:run
```

After running the app, we can call the `/wallet` endpoint to calculate our stock portfolio.

```
curl http://localhost:8080/wallet
```

Here's the response returned by OpenAI for the provided test data and the current stock market prices.

```
~ (13.013s)
curl http://localhost:8080/wallet
Here are the latest stock prices and the value of your wallet based on your shares:

- **AAPL**:: 100 shares at $238.60 each = $23,860.00
- **AMZN**:: 300 shares at $235.84 each = $70,752.00
- **META**:: 300 shares at $702.23 each = $210,669.00
- **MSFT**:: 400 shares at $416.49 each = $166,596.00
- **NVDA**:: 200 shares at $121.56 each = $24,312.00

## Total Value of Wallet:
- **Total** = $23,860.00 + $70,752.00 + $210,669.00 + $166,596.00 + $24,312.00 = **$496,189.00**

Your current wallet value is **$496,189.00**.
```

Then, let's switch to the application logs. We can see that the `StockService` function was called five times – once for every company in the wallet. After we added the `SimpleLoggerAdvisor` advisor to the and set the property `logging.level.org.springframework.ai` to `DEBUG`, we can observe detailed logs with requests and responses from the OpenAI chat model.

```
2025-01-30T16:32:49.333+01:00 DEBUG 22318 --- [nio-8080-exec-1] o.s.a.c.c.advisor.SimpleLoggerAdvisor : request: AdvisedRequest[chatModel=OpenAIChatModel [defaultOptions=OpenAIChatOptions: {"streamUsage":false,"model":"gpt-4o-mini","temperature":0.7}], userText=What's the current value in dollars of my wallet based on the latest stock daily prices ?
, systemText=null, chatOptions=OpenAIChatOptions: {"streamUsage":false, media=[], functionNames=[], functionCallbacks=[], messages=[], userParameters={}, systemParameters={}, advisors=[org.springframework.ai.chat.client.DefaultChatClient$DefaultChatClientRequestSpec$@4dc4040a, org.springframework.ai.chat.client.DefaultChatClient$DefaultChatClientRequestSpec$@47e26bd0, SimpleLoggerAdvisor, org.springframework.ai.chat.client.DefaultChatClient$DefaultChatClientRequestSpec$@3c2f8fbb], advisorParameters={}, adviseContext={}, toolContext={}]
2025-01-30T16:32:55.562+01:00 INFO 22318 --- [nio-8080-exec-1] p.p.s.functions.stock.StockService : Get stock prices: AAPL -> 238.5
2025-01-30T16:32:55.616+01:00 INFO 22318 --- [nio-8080-exec-1] p.p.s.functions.stock.StockService : Get stock prices: AMZN -> 235.27
2025-01-30T16:32:55.670+01:00 INFO 22318 --- [nio-8080-exec-1] p.p.s.functions.stock.StockService : Get stock prices: META -> 701.68
2025-01-30T16:32:55.720+01:00 INFO 22318 --- [nio-8080-exec-1] p.p.s.functions.stock.StockService : Get stock prices: MSFT -> 414.59
2025-01-30T16:32:55.773+01:00 INFO 22318 --- [nio-8080-exec-1] p.p.s.functions.stock.StockService : Get stock prices: NVDA -> 119.11
2025-01-30T16:33:02.360+01:00 DEBUG 22318 --- [nio-8080-exec-1] o.s.a.c.c.advisor.SimpleLoggerAdvisor : response: {"metadata":{"id":"chatcmpl-AvQpEBAarkdt6t6cFKW0W6s6t1Hw","tokensLimit":200000,"tokensRemaining":19977,"tokensReset":0.034000000,"usage":{"promptTokens":561,"generationTokens":340,"totalTokens":901},"promptMetadata":{"empty":false},"results":{"metadata":{"finishReason":"STOP","contentFilters":{},"empty":true},"output":{"messageType":"ASSISTANT","metadata":{"refusal":"","finishReason":"STOP","index":0,"rule":"ASSISTANT","id":"chatcmpl-AvQpEBAarkdt6t6cFKW0W6s6t1Hw","messageType":"ASSISTANT"},"toolCalls":[],"media":[],"content":"Here are the latest stock prices for the companies in your portfolio:\n\n- **AAPL**:: $238.5
```

Final Thoughts

In this article, we analyzed the Spring AI integration with function support in AI models. OpenAI's function calling is a powerful feature that enhances how AI models interact with external tools, APIs, and structured data. It makes AI more interactive and practical for real-world applications. Spring AI provides a flexible way to register and invoke such functions. However, it still requires attention from developers, who need to define clear function schemas and handle edge cases.

AI

LLM

open-ai

Spring Boot

Spring Functions

spring-ai

spring-ai-function-calling



Written by:

piotr.minkowski

[View All Posts →](#)

6 COMMENTS



Learner January 30, 2025 11:38 pm

How openai know about your api and how were you able to run it on local while open ai need public api? Can you please elaborate.

Loading...

[REPLY](#)



piotr.minkowski February 4, 2025 10:12 am

OpenAI doesn't call your API over internet. Instead, it just suggests function calls, and your code executes them locally.

Your Spring Boot app sends a request to OpenAI with function definitions. Then OpenAI responds with the function name and parameters. Your app receives the response and runs the function locally. More details: <https://docs.spring.io/spring-ai/reference/api/chat/functions/openai-chat-functions.html>

Loading...

[REPLY](#)



seckinkarabel February 2, 2025 11:35 am

Thank you for this great article and keep it up coming...

Just a quick note: I found mistakes on the architecture section, you mixed the usage of StockService with WalletService both in the text and architecture image.

I should be written:

Chat model should call StockService function for each object in the list returned by WalletService function

Loading...

[REPLY](#)



piotr.minkowski February 4, 2025
10:20 am

Thanks for careful reading. I changed it as you suggested

Loading...

[REPLY](#)



Lucien Liu March 2, 2025 2:25 am

Could the function run with ollama? How to change it to Ollama?

I run with ollama raise the error

[404] Not Found – {"error": "model \\'mxbai-embed-large\' not found, try pulling it first"}

Loading...

[REPLY](#)



piotr.minkowski March 3, 2025 12:51 pm

I didn't try it with ollama. According to the Spring AI docs:

https://docs.spring.io/spring-ai/reference/api/chat/functions/ollama-chat-functions.html#_how_it_works – it should work. Anyway, a functional calling feature is already deprecated in Spring AI... I'll update this article to describe replacement – Tool Calling

Loading...

[REPLY](#)

Leave a Reply

Contact info

If you would like to contact me in order you have any questions, thoughts or ideas (e.g. suggestions for future articles) contact me via email.

♥ Warsaw, Poland
piotr.minkowski@gmail.com

Top Posts & Pages

- [Claude Code Template for Spring Boot](#)
- [Startup CPU Boost in Kubernetes with In-Place Pod Resize](#)
- [Part 1: Creating microservice using Spring Cloud, Eureka and Zuul](#)

Social Media

- [Create Apps with Claude Code on Ollama](#)
- [Deep Dive into Kafka Offset Commit with Spring Boot](#)